

生成 AI 利用申告書

25G1053 齋藤 翔太

2026 年 4 月 27 日

プログラムファイル名: HCK02_02.ino, HCK02_02.pde

使用した AI ツール: Gemini

1 何に使ったか

今回の課題において、Arduino からのバイナリ通信 (`Serial.write()`) の実装、および Processing 側でのシリアル通信を用いた波形描画プログラムの作成に使用した。また、実行時に発生したクラッシュエラー (`null` エラー) の原因特定と解決策の検討にも使用した。

2 AI への指示 (プロンプト)

- 「Arduino から processing に取得した波形のデータを送る方法についてヒントをください。」
- 「実行したところシリアルモニターには「？」が羅列されて processing の方は真っ黒のままでした。原因は为什么呢か。」
- 「processing 内のコンソールに、Error, disabling serialEvent() for /dev/cu.usbmodem34B7DA6448002 null という文がでました。」

3 AI の出力の使い方

AI から提案された `while (port.available() > 0)` を用いた受信バッファの処理や、`map()` 関数による画面サイズに合わせた座標変換の仕組みを理解した上で、自身の Mac 環境のポート名 (`/dev/cu.usbmodem34B7DA6448002`) に合わせてプログラムを修正して組み込んだため。

4 正しいと確認した方法

- 修正したプログラムを実行し、作成した楽曲を入力して動作を確認した。
- Processing のウィンドウ上に、緑色の波形が連続して途切れることなく描画されることを目視

で確認し、コードの意図通りに動作していることを検証した。

5 問題や間違い

初期のプログラムでは、データ受信用の `serialEvent()` 内で線の描画や背景の更新を行っていたが、数十個のデータを受け取った直後に `null` エラーが発生し通信が強制終了する問題に直面した。エラーメッセージを基に確認した結果、Processing 4 の仕様変更により、通信用の裏側のプロセスから直接画面を描画できないことが原因であると特定できたため、描画処理をメインの `draw()` ループ内に移す修正を行った。

6 自分で工夫した点

- Arduino 側の処理負荷を下げるため、計算した電圧 (float 型) をそのまま送るのではなく、AD 変換値 (0~1023) を単純に 4 で割って 1 バイト (0~255) のバイナリデータとして `Serial.write()` で高速送信する方式を採用した。
- エラーを回避しつつ高速な通信を取りこぼさないよう、描画処理をメインループである `draw()` 内に移動させ、`while` 文を用いて受信バッファにあるデータを一気に処理してから波形を描く安全な構造に改修した。
- 波形の推移を視覚的に分かりやすくするため、黒背景に緑線を描くようにし、X 座標の進み具合 (`x += 2`) を調整して波の間隔を最適化した。

参考文献

- [1] 有本泰子. (2026). Processing の使い方.
- [2] 有本泰子, & 佐波孝彦. (2026). ハッカソン 1 第 2 回 講義資料.
- [3] はてなブログ.Processing のシリアル通信で「Error, disabling serialEvent()」というエラーがでたら.[\(https://kougaku-navi.hatenablog.com/entry/2019/10/10/153000\)](https://kougaku-navi.hatenablog.com/entry/2019/10/10/153000).(2026/04/27)
- [4] htsuda.net. 線の色と太さ: `stroke`, `noStroke`, `strokeWeight`. [\(https://htsuda.net/archives/1337\)](https://htsuda.net/archives/1337).(2026/04/27)